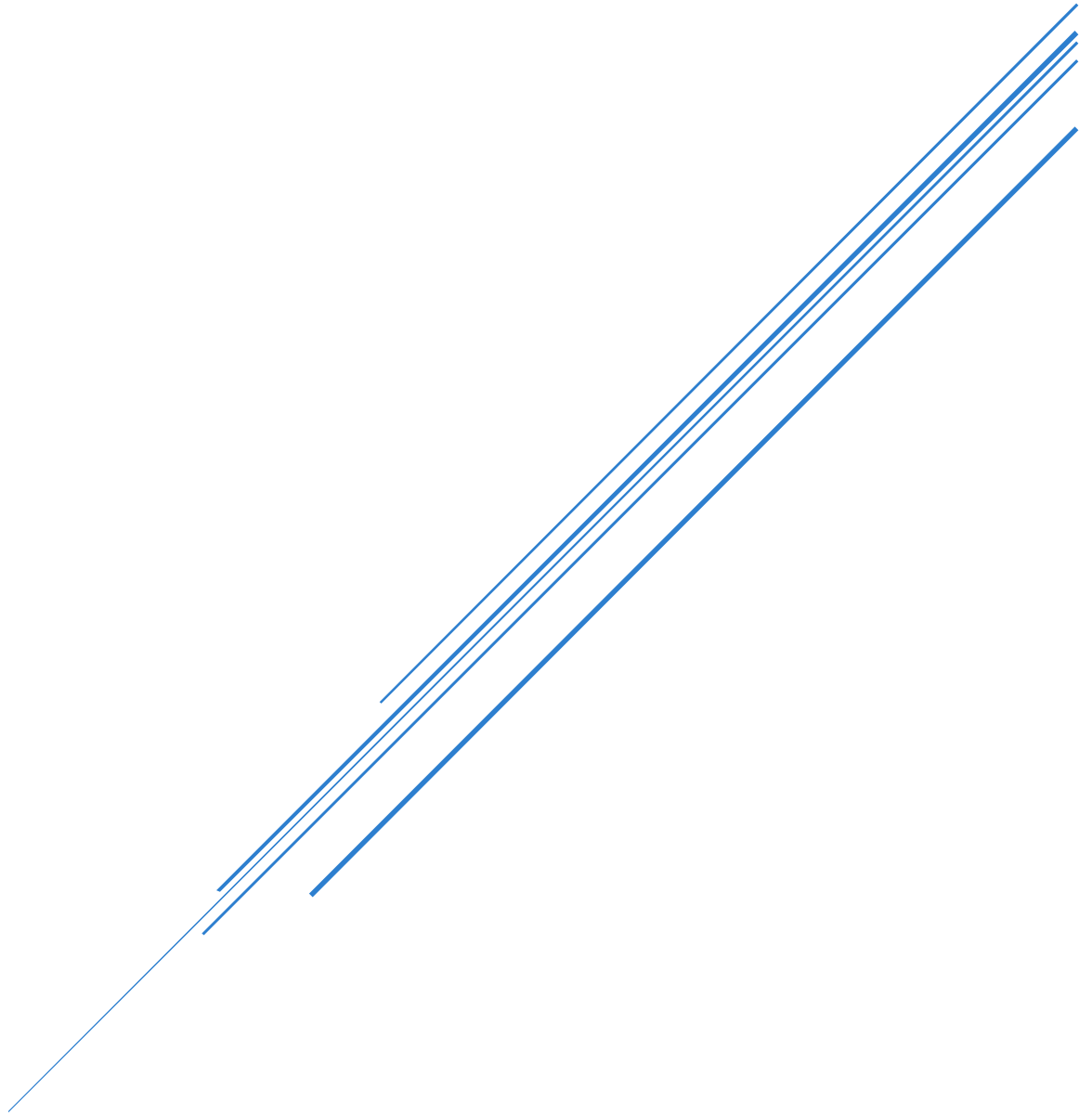


DESIGN PROJECT 2 PT.3

Microprocessors

5 May 2025



Annabel Gabriel, Tyler Giczkowski, Aidan VandenElzen
COEN 2710

Summary:

For design project 2, part 3, the team was tasked to simulate a RISC-V processor using a top-level design implementation to instantiate all of the components within the processor. This simulation was created in Vivado in the language VHDL. The processor has been successfully tested utilizing test benches and waveform charts to monitor proper input-output functions. For each component, using Vivado's report timing summary, the longest timing delay can be determined. For the RISC-V processor, the most critical path showed a timing delay of 16.283ns.

Description:

RISC-V Processor:

The RISC-V Processor has multiple components that perform different tasks to allow data and control signals to flow. The PC (program counter) tracks the address of the next instruction to execute. The instruction memory stores the program instructions, with the current instruction fetched using the PC address. The control unit decodes fetched instructions and generates control signals for various modules like ALU, multiplexers, and memory. The signals in this component include a clock, reset, Branch, MemRead, ALUOp, etc. The register file houses registers for storing intermediate values. It supports read/write operations for instruction execution. The imm gen (immediate generation) extracts and formats immediate values from instruction fields. The ALU (Arithmetic Logic Unit) performs computation tasks, such as arithmetic operations, logic functions, and comparisons. Inputs come from registers or immediate values. The data memory serves as storage for data accessed during load/store instructions. The Muxes (Multiplexers) direct appropriate input data to other components like the ALU or PC, depending on control signals. Finally, the adders are used to calculate sequential PC updates or branch target addresses.

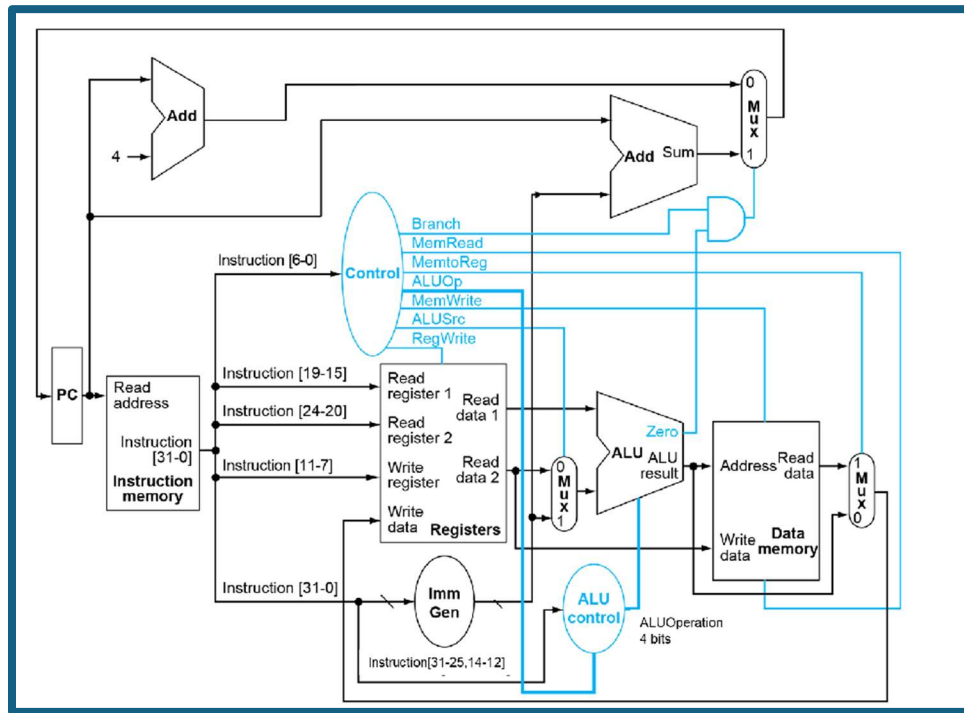


Figure 1: Top-Level Block Design of the Data Path

Each component in the Data Path shown above in Figure 1 was designed and implemented in VHDL. Each component was tested in parts to make sure that the individual components themselves were working properly. To complete the RISC-V Processor, a top-level design was used to connect all the corresponding components to their correct inputs and outputs, essentially wiring the components together to match the design of the Data Path above. While every component is essential, there are a few components that need to be stressed more when implementing this top-level design. Specifically, attention should be given to the instruction memory, as this is where the instructions in RISC-V are given and read to follow through the data Path. The registers, as this is where data is read to and from and sent to the ALU for operations and the data memory, where the addresses and data is read. Finally, the third mux, in the bottom left of Figure 1, which gives the final output to be read back to the registers. The final schematic and implementation for the Top-Level Data Path is depicted in Figure 2.

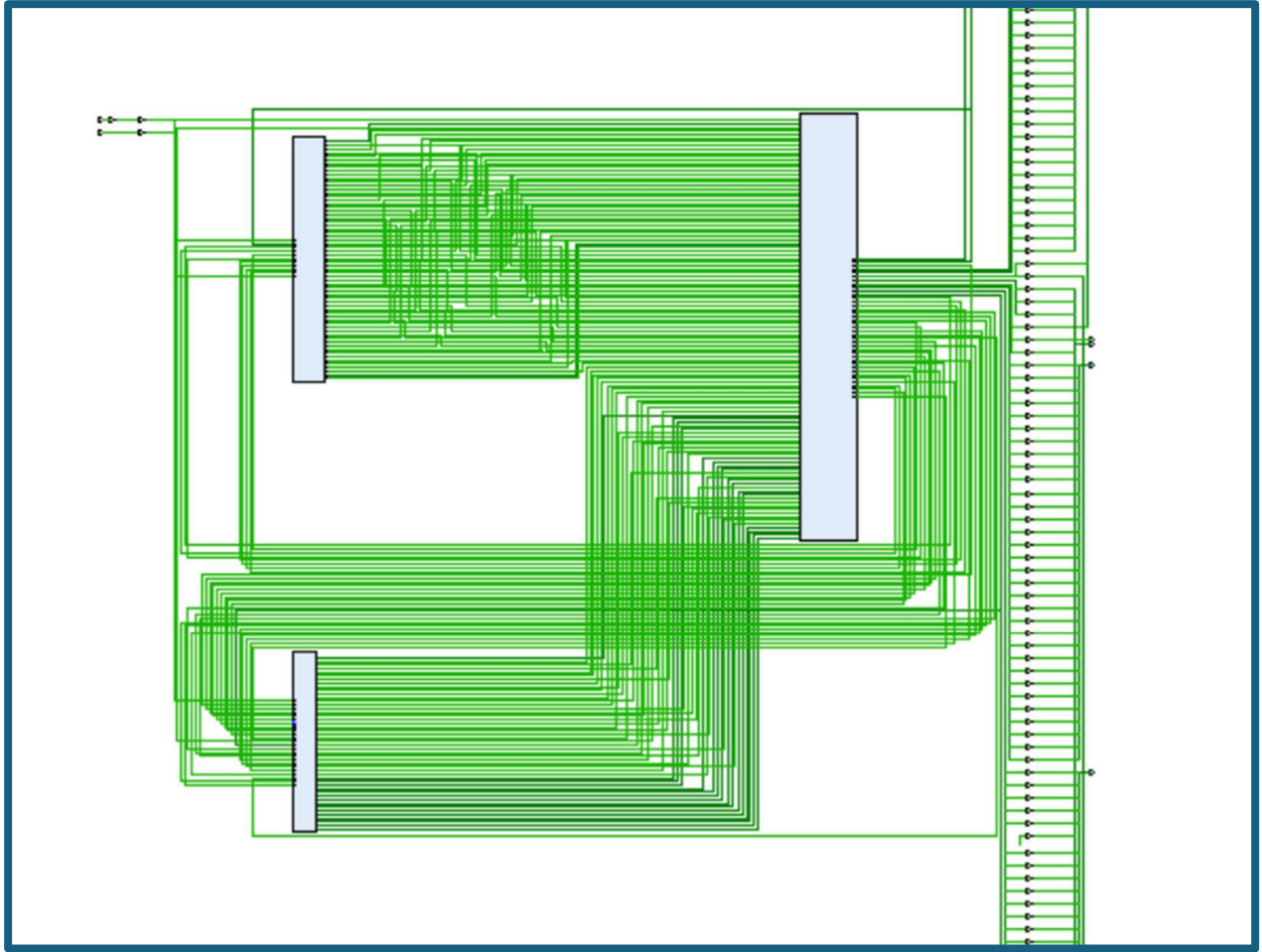


Figure 2: Central Portion Schematic of the Top-Level Data Path.

Testing and Verification with Results:

RISC-V Processor:

The RISC-V Processor was tested using a testbench depicted in Figure 12. The inputs to the testbench were to test this instruction set depicted below in Table 1.

Table 1: RISC-V Instructions and Their Respective 32-Bit Hexadecimal Conversion

RISC-V Instruction	Hexadecimal Conversion
lw x6, 10(x0)	00A02303
lw x7, 11(x0)	00B02383
lw x28, 13(x0)	00D02E03
add x29, x6, x7	00730EB3
add x30, x6, x7	00730F33
BEQ x28, x29, 2	01DE0263
add x30, x28, x29	01DE0F33
sw x29, 14(x0)	01D02723
lw x31, 14(x0)	00E02F83

From 0ns to 70ns, the reset signal is active and therefore the outputs that should be referenced, Mux 3 and regFiles, are set to 0. The timing diagram for this time reference is shown in Figure 3.

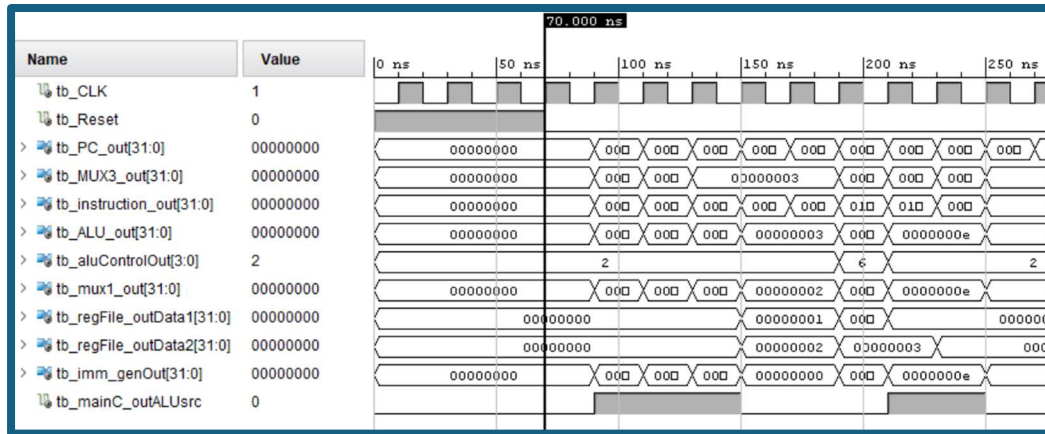


Figure 3: Timing Diagram of the RISC-V Processor from 0-70ns.

At 90ns, the first rising edge of the clock occurs where “Reset” is not active, this begins the first instruction lw x6, 10(x0) (00A02303). This is an I-type instruction, loading the value in 10(x), which is equal to 1, into register x6. Our expected output, passed to Mux 3 is 1, which is what is seen in Figure 4.

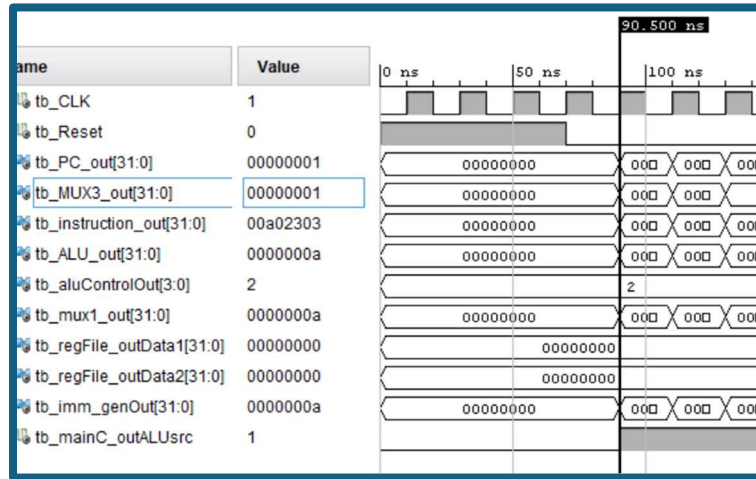


Figure 4: Timing Diagram of the RISC-V Processor from 90-110ns.

From 110ns to 130ns, the second rising clock edge occurs where “Reset” is not active, this begins the second instruction `lw x7, 11(x0)` (00730F33). This is an I-type instruction, loading the value in 11(x), which is equal to 2, into register x7. Our expected output passed to Mux 3 is 2, which is what is seen in Figure 5.

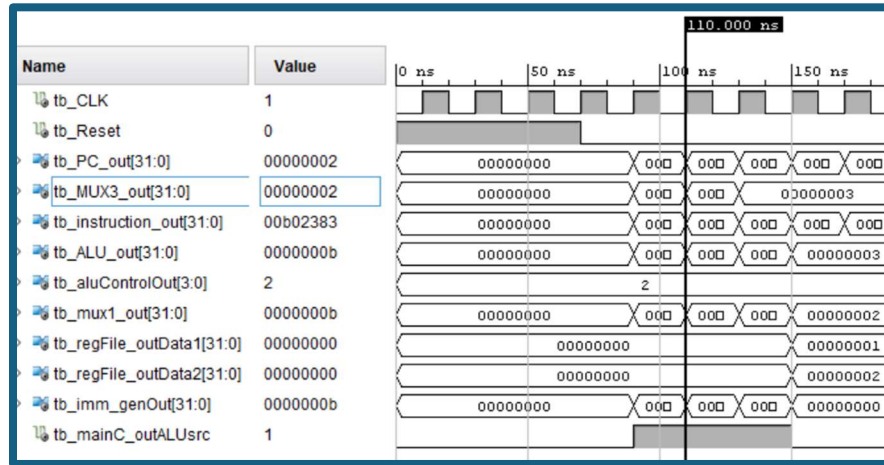


Figure 5: Timing Diagram of the RISC-V Processor from 110-130ns.

From 130ns to 150ns, the third rising clock edge occurs where “Reset” is not active, this begins the third instruction `lw x28, 13(x0)` (00D02E03). This is an I-type instruction, loading the value in 13(x), which is equal to 3, into register x28. Our expected output passed to Mux 3 is 3, which is what is seen in Figure 6.

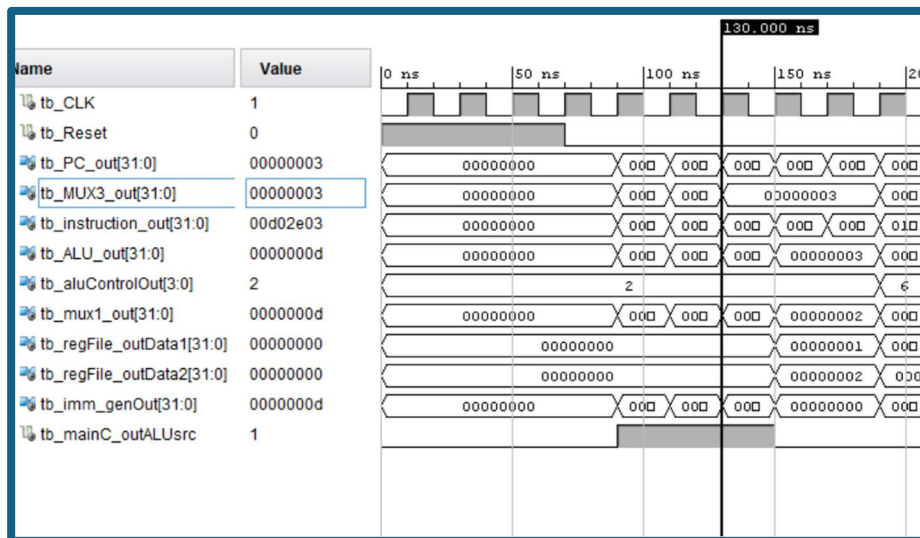


Figure 6: Timing Diagram of the RISC-V Processor from 130-150ns.

From 150ns to 170ns, the fourth rising clock edge occurs where “Reset” is not active, this begins the fourth instruction add x29, x6, x7 (00730EB3). This is an R-type instruction, adding the loaded values of registers x6 and x7, 1 and 2 respectively, into register x29. Our expected output passed to Mux 3 is 3, which is what is seen in Figure 7.

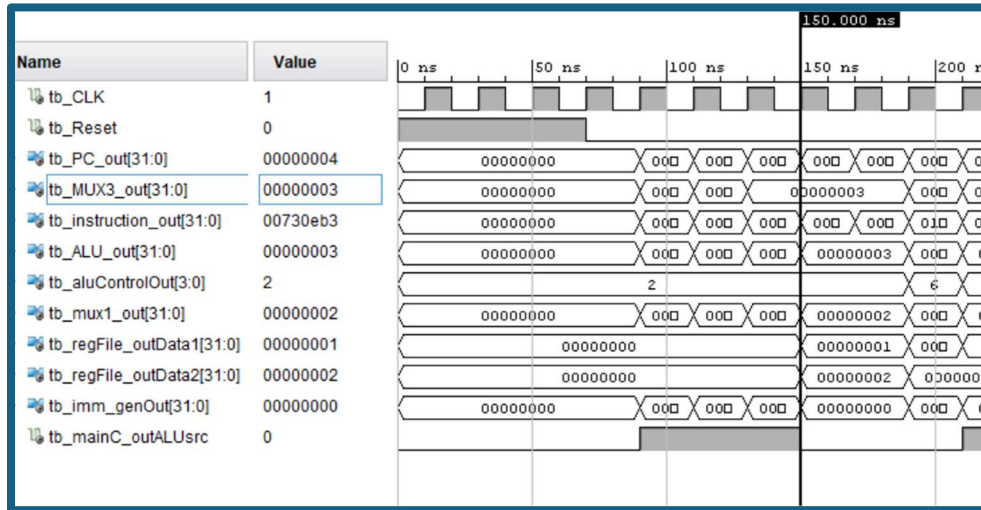


Figure 7: Timing Diagram of the RISC-V Processor from 150-170ns.

From 170ns to 190ns, the fifth rising clock edge occurs where “Reset” is not active, this begins the fifth instruction add x30, x6, x7 -> $x30 = 1 + 2 = 3$ (00730F33). This is an R-type instruction, adding the loaded values of registers x6 and x7, 1 and 2 respectively, into register x30. Our expected output passed to Mux 3 is 3, which is what is seen in Figure 8.

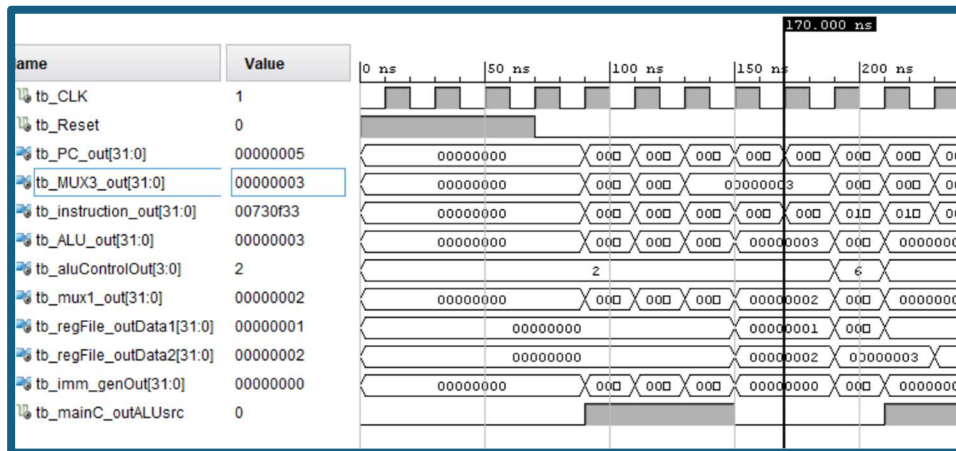


Figure 8: Timing Diagram of the RISC-V Processor from 170-190ns.

From 190-210ns, the sixth rising clock edge occurs where “Reset” is not active, this begins the sixth instruction BEQ x28, x29, 2 (01DE0263). This conditional checks to see if the data stored in registers x28, x29 are equal and if they are, to branch to the instruction where PC=8. Since x28 and x29 both equal 3, this will branch the instructions to PC=8. The expected output is 0 as to perform a BEQ instruction the values in x28 and x29 are subtracted to find equivalence, which is seen in Figure 9.

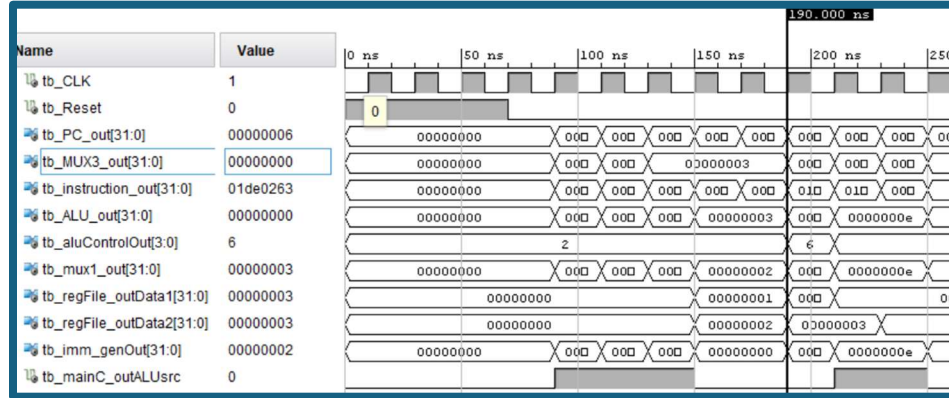


Figure 9: Timing Diagram of the RISC-V Processor from 190-210ns.

From 210-230ns, the seventh rising edge of the clock occurs where “Reset” is not active, this begins the eighth instruction sw x29, 14(x0) (01D02723). This is an S-type instruction, storing the value in x29, which is 3, into memory location 14(x0). Our expected output, passed to reg_file_outdata2 is 3, which is what is seen in Figure 10.

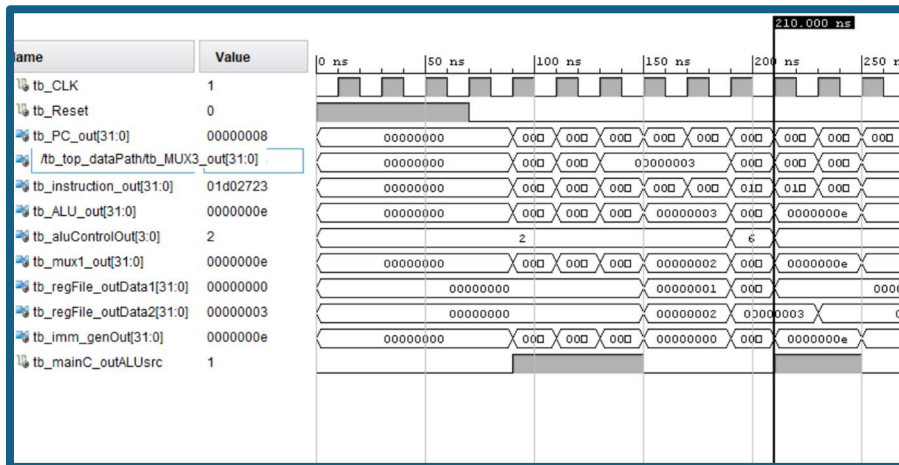


Figure 10: Timing Diagram of the RISC-V Processor from 210-230ns.

From 230-250ns, the eighth rising clock edge occurs where “Reset” is not active, this begins the ninth instruction lw x31, 14(x0) (00E02F83). This is an I-type instruction, loading the value in 14(x), which is equal to 3, into register x31. Our expected output passed to Mux 3 is 3, which is what is seen in Figure 11.

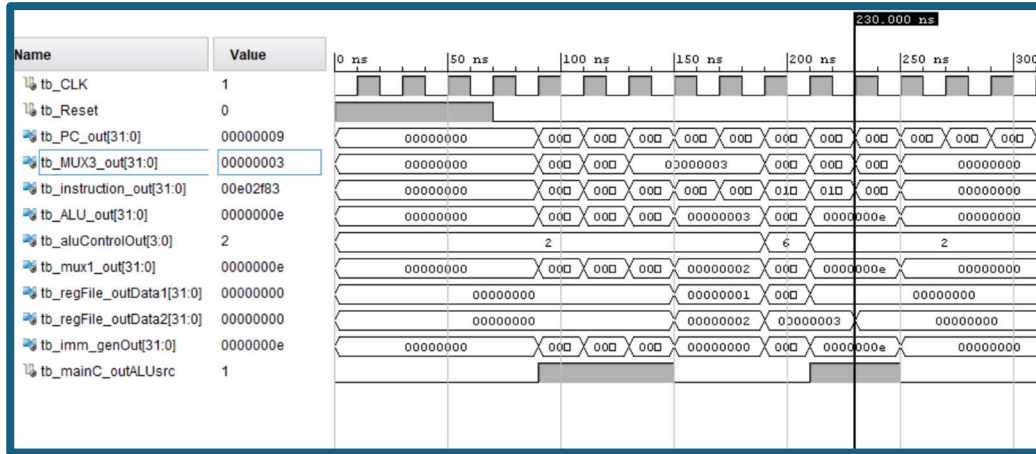


Figure 11: Timing Diagram of the RISC-V Processor from 230-250ns.

The total testbench from 0-300ns is shown below in Figure 12, with every signal and respective outputs. The passing of instructions is illustrated clearly with the rising clock edges. As well, each component of the RISC-V Processor can be seen on the left side with its outputs to the right of it.

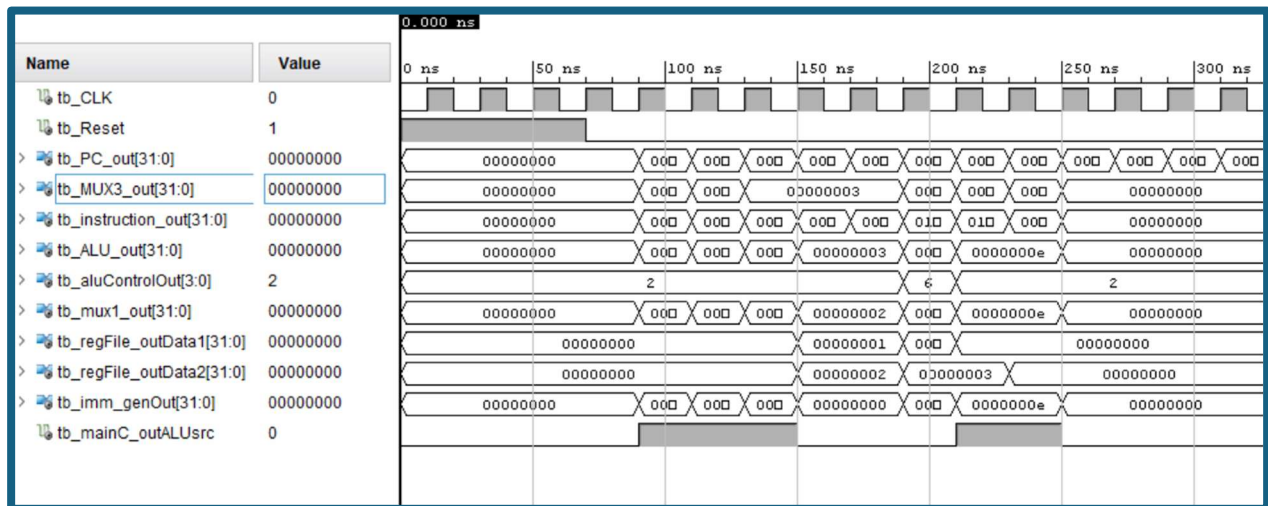


Figure 12: Test bench simulation for the RISC-V Processor.

Performance Analysis:

RISC-V Processor:

The timing analysis for the RISC-V Processor revealed the longest timing delay for the circuit was from the input PC (Program Counter) to the output Mux3. The total delay of this path was 16.283ns. Figure 13 demonstrates this by showing only the most-critical path that this delay time references, starting at PC, and ending at Mux3.

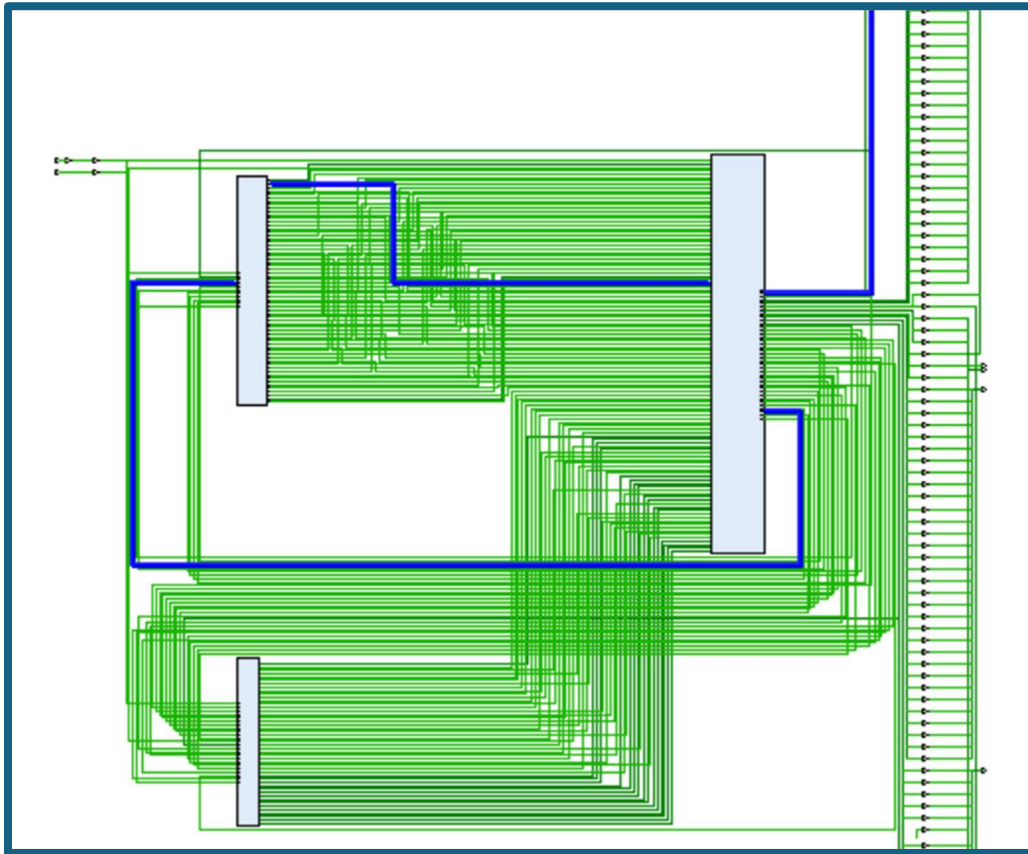


Figure 13: Schematic diagram depicting the longest path by the blue line.

Conclusions:

The purpose of this project was to design, simulate, and test a RISC-V processor using a top-level implementation to instantiate its components. The design and simulation were carried out using VHDL in Xilinx Vivado, with testbenches and waveform analysis employed to validate the processor's functionality. Each component, including the program counter (PC), control unit, ALU, and multiplexers, was implemented and tested independently before integrating them into the full processor design.

The team utilized timing reports from Vivado to analyze critical paths, with the most critical path exhibiting a delay of 16.283 ns. The project emphasized translating the processor's data path diagrams into VHDL descriptions, leveraging guidance from the textbook DIGITAL DESIGN USING DIGILENT FPGA BOARDS: Vhdl (Haskell, R., 2017). Through this process, the team gained a deeper understanding of digital design principles and the timing considerations inherent in processor design.

Challenges were encountered, including initial synchronization issues when integrating individual components and debugging unexpected simulation results. These were addressed through collaboration, detailed analysis, and iterative testing. Paired programming proved particularly effective in resolving issues related to logic errors and hidden syntax mistakes. Ultimately, the project provided valuable insights into the intricacies of microprocessor design and the practical application of VHDL in hardware simulation and verification.

References:

- [1] *Installer Information*. (n.d.). AMD. <https://www.xilinx.com/support/installer/installer-info-general.html>
- [2] Ababei, C. *Arithmetic for Computers – Building the ALU (Ch.3 + Appendices A,C)*
- [3] Haskell, R. (2017). *DIGITAL DESIGN USING DIGILENT FPGA BOARDS: Vhdl*.